CodInClub
Powered By BridgeLabz

What is a Constructor?

A constructor in C# is a special method that initializes an object when it is created. Unlike regular methods, constructors:

- Have the same name as the class.
- Do not have a return type.
- Are automatically called when the object is instantiated using the new keyword.

Key Features of Constructors

- **Automatic Invocation:** Called during object creation.
- **Overloading Support:** You can define multiple constructors with different parameter lists.
- **Initialization:** Primarily used to initialize object attributes.

Coffee Shop Order System

Let's create a CoffeeShopOrder class to manage customer orders in a coffee shop.

Sample Program: CoffeeShopOrder Class

```
Java
using System;

class CoffeeShopOrder
{
    // Attributes
    public string CustomerName { get; set; }
    public string CoffeeType { get; set; }
    public int Quantity { get; set; }
    public double TotalPrice { get; private set; }

    // Default Constructor
    public CoffeeShopOrder()
    {
        CustomerName = "Guest";
        CoffeeType = "Regular";
        Quantity = 1;
        TotalPrice = CalculatePrice();
    }

    // Parameterized Constructor
```

```
    public CoffeeShopOrder(string customerName, string coffeeType, int
quantity)
    {
        CustomerName = customerName;
        CoffeeType = coffeeType;
        Quantity = quantity;
        TotalPrice = CalculatePrice();
    }

    // Copy Constructor
    public CoffeeShopOrder(CoffeeShopOrder previousOrder)
    {
        CustomerName = previousOrder.CustomerName;
        CoffeeType = previousOrder.CoffeeType;
        Quantity = previousOrder.Quantity;
        TotalPrice = previousOrder.TotalPrice;
    }

    // Method to calculate price
    private double CalculatePrice()
    {
        double pricePerCup = CoffeeType.ToLower() switch
        {
            "latte" => 5.0,
            "espresso" => 4.0,
            "cappuccino" => 4.5,
            _ => 3.0 // Regular coffee
        };
        return pricePerCup * Quantity;
    }

    // Display Order Details
    public void DisplayOrderDetails()
    {
        Console.WriteLine($"Customer Name: {CustomerName}\nCoffee Type:
{CoffeeType}\nQuantity: {Quantity}\nTotal Price: ${TotalPrice}");
    }
}

class Program
{
    static void Main()
    {
        // Default Constructor
        CoffeeShopOrder order1 = new CoffeeShopOrder();
        Console.WriteLine("Order 1:");
        order1.DisplayOrderDetails();

        // Parameterized Constructor
```

```
        CoffeeShopOrder order2 = new CoffeeShopOrder("Alice", "Latte", 3);
        Console.WriteLine("\nOrder 2:");
        order2.DisplayOrderDetails();

        // Copy Constructor
        CoffeeShopOrder order3 = new CoffeeShopOrder(order2);
        Console.WriteLine("\nOrder 3 (Copy of Order 2):");
        order3.DisplayOrderDetails();
    }
}
```

Output:

Order 1:
Customer Name: Guest
Coffee Type: Regular
Quantity: 1
Total Price: $3.0

Order 2:
Customer Name: Alice
Coffee Type: Latte
Quantity: 3
Total Price: $15.0

Order 3 (Copy of Order 2):
Customer Name: Alice
Coffee Type: Latte
Quantity: 3
Total Price: $15.0

Explanation of the Program

1. **Default Constructor**:
   - Initializes attributes with default values.
   - In this example, if no parameters are provided, the customer is treated as a "Guest" ordering one "Regular" coffee.
2. **Parameterized Constructor**:
   - Accepts values for `CustomerName`, `CoffeeType`, and `Quantity`.
   - Dynamically calculates the total price based on the coffee type and quantity.
3. **Copy Constructor**:
   - Creates a new object by copying the attributes of an existing object.

- Used here to duplicate an order for simplicity.

4. **Switch Case for Coffee Prices**:
   - Dynamically assigns the price per cup based on the coffee type.

Constructor Overloading in Action

The program demonstrates constructor overloading by using multiple constructors:

1. The **default constructor** initializes default values.
2. The **parameterized constructor** customizes the initialization.
3. The **copy constructor** replicates an existing object's attributes.

Access Modifiers in C#

What are Access Modifiers? Access modifiers in C# control the visibility and accessibility of classes, methods, and attributes.

Modifier	Class	Same Assembly	Derived Class	Outside Assembly
public	Yes	Yes	Yes	Yes
protected	Yes	Yes	Yes	No
internal	Yes	Yes	No	No
private	Yes	No	No	No

Example:

```java
class Example
{
    public int PublicVariable = 10;       // Accessible everywhere
    protected int ProtectedVariable = 20; // Accessible in the same assembly
and subclasses
    internal int InternalVariable = 30;   // Accessible within the same
assembly
    private int PrivateVariable = 40;     // Accessible only within the
class

    public void DisplayAccess()
```

```
    {
        Console.WriteLine($"Public: {PublicVariable}");
        Console.WriteLine($"Protected: {ProtectedVariable}");
        Console.WriteLine($"Internal: {InternalVariable}");
        Console.WriteLine($"Private: {PrivateVariable}");
    }
}
```

Instance vs. Class Variables and Methods

Instance Variables and Methods

Instance Variables:

- Belong to an object; unique to each instance.
- Example: string Name;

Instance Methods:

- Operate on instance variables and can be accessed only via objects.
- Example:

```
void DisplayDetails() {
   Console.WriteLine(Name);
}
```

Class Variables and Methods

Class Variables:

- Declared using the static keyword. Shared among all objects.
- Example:

```
static int totalStudents;
```

Class Methods:

- Operate on static variables and can be accessed without creating an object.
- Example:

```
static void IncrementStudents() {
   totalStudents++;
}
```

Aspect	Instance Variables/Methods	Class Variables/Methods
Belongs To	A specific object	The class itself
Access	Through object	Through class name or object
Keyword	No special keyword	Declared using static
Example	student.Name	Student.TotalStudents

Sample Program:

```java
using System;

class School
{
    public string StudentName;        // Instance Variable
    public static int TotalStudents;  // Class Variable

    // Constructor
    public School(string studentName)
    {
        StudentName = studentName;
        TotalStudents++;       // Increment total students
    }

    // Instance Method
    public void DisplayStudent()
    {
        Console.WriteLine("Student Name: " + StudentName);
    }

    // Class Method
    public static void DisplayTotalStudents()
    {
        Console.WriteLine("Total Students: " + TotalStudents);
    }
}

class Program
{
    static void Main()
    {
        School student1 = new School("Alice");
```

```
        School student2 = new School("Bob");

        student1.DisplayStudent();
        student2.DisplayStudent();

        School.DisplayTotalStudents();  // Accessing class method
    }
}
```

Output:

Student Name: Alice

Student Name: Bob

Total Students: 2